

Technical Architecture Document

Project: Prism AI | Team Jude Force | SIH26154

1. High-Level Architecture

Prism AI is a layered, pipeline-style system: a React dashboard calls a FastAPI backend, which hands work to Celery workers. Workers run input guardrails, parsing, RAG, LoRA-based generation and output guardrails, using PostgreSQL, MinIO, Qdrant and Redis as local services.

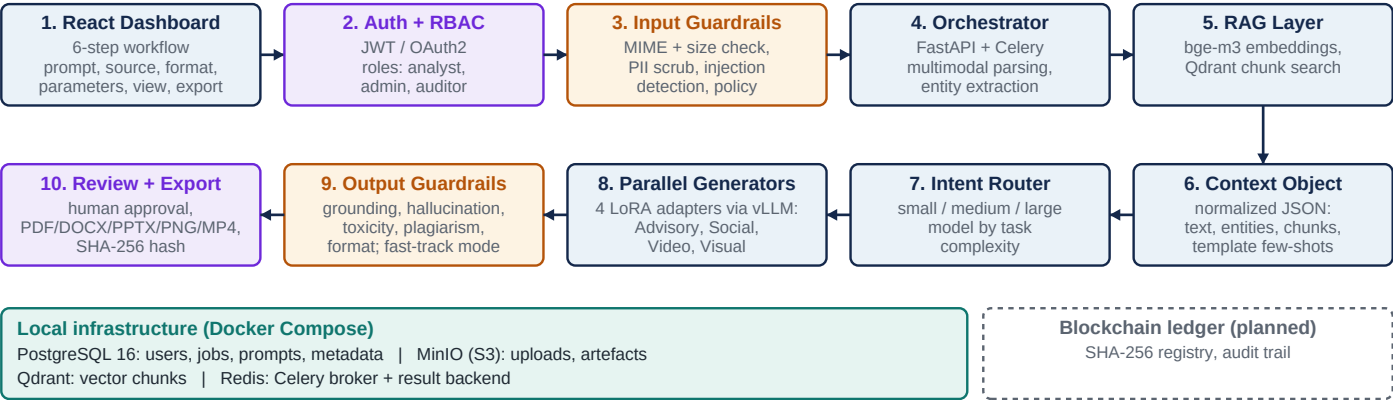


Figure 1. Request flow: User → Frontend → API → guardrails and pipeline → storage and AI services → export. Dashed box: planned.

2. Technology Stack

Layer	Technologies
Frontend	React (Vite), Zustand, Axios, React Query, react-dropzone
Backend	Python 3.11, FastAPI, Celery, Redis, SQLAlchemy, Alembic
AI / ML	Qwen2.5-7B-Instruct, 4-bit QLoRA (Unsloth, Hugging Face PEFT, TRL), vLLM, BAAI/bge-m3 (sentence-transformers)
Parsing	PyMuPDF, pdfplumber, python-docx, Pytesseract, faster-whisper, spaCy, langdetect
Guardrails	Presidio Analyzer + Anonymizer, Detoxify, RapidFuzz, keyword-overlap checks, hashlib (SHA-256)
Export	ReportLab (PDF), python-docx (DOCX), python-pptx (PPTX)
Data / Infra	PostgreSQL 16, Qdrant, MinIO (S3-compatible), Redis 7, Docker Compose

3. Major System Components

Component	Role
React Dashboard	6-step operator workflow: Prompt, Attach Source, Select Format, Parameters, View Output, Export.
Auth + RBAC	JWT / OAuth2 with roles (analyst, admin, auditor).
Input Guardrails	File validation, PII redaction, prompt-injection and policy checks before content reaches the LLM.
Orchestrator	FastAPI endpoints plus Celery tasks; multimodal parsing and Normalized Context Object creation.
RAG Layer	Embeds chunks with bge-m3 and retrieves relevant context from Qdrant per job.
Intent Router	Chooses small, medium or large model by task complexity.
Generators	Parallel format generation on four LoRA adapters (Advisory, Social, Video, Visual) served by vLLM.
Output Guardrails	Grounding, hallucination, toxicity, plagiarism and format checks on each artefact.
Review + Export	Human approval, SHA-256 hashing, export to PDF / DOCX / PPTX; PNG and MP4 outputs are listed in the export dashboard.

4. Data / Request Flow

Step	What happens
1. Upload	POST /upload: validate_file() checks type (pdf, docx, txt, jpg, png, mp3, mp4) and size (max 50 MB); file stored in MinIO; SourceContent row created.
2. Generate	POST /generate creates a Job (status queued) and calls process_generation_job.delay().
3. Parse + guard	Worker sets processing, runs parse_source(), then PII redaction and injection detection. Injection detected: job failed, no embedding.
4. Index	Text is chunked (~200–400 words) and embedded into the Qdrant collection with job_id as payload.
5. Generate	Top-k chunks (k=5) + prompt + parameters go to the selected LoRA adapter, or a prompt-engineered fallback; one GeneratedArtefact per format.
6. Check + export	Output checks run, SHA-256 is stored, and GET /export/{id} returns MinIO presigned download links. UI polls GET /jobs/{id}.

5. Component-Level Architecture

Module (backend / ai)	Responsibility
app/core/	config.py (pydantic-settings), database.py (SQLAlchemy session), minio_client.py (bucket check, upload; logs a warning instead of crashing if MinIO is down), integrity.py (SHA-256).
app/api/routes.py	/upload, /generate, /jobs/{id}, /export/{id}.
app/worker.py, tasks.py	Celery app (Redis broker and result backend); process_generation_job(job_id) runs the whole pipeline.
app/pipeline/	multimodal_parser.py, rag_retriever.py (embed_and_store, retrieve_relevant_context), export_formatter.py.
app/guardrails/	input_guardrails.py and output_guardrails.py (see Section 10).
ai/	training/ (synthetic data, LoRA training), adapters/ (trained adapters), serving/vllm_server.py (adapter hot-swap).

6. Frontend Architecture

- Single-page React app built with Vite; App.jsx renders a step indicator and the step component selected by currentStep.
- **State:** Zustand store (step, prompt, source id and name, output formats, generation parameters, job id and status). **API:** Axios client with base URL from VITE_API_URL. **Status:** React Query polls /jobs/{id} every 2 s and stops on *completed* or *failed*.
- **Inputs:** prompt, file upload (react-dropzone), format multi-select, tone / audience / language / level of detail / template. Output preview and export follow in steps 5 and 6.

7. Backend Architecture

A stateless FastAPI service (uvicorn) with CORS restricted to ALLOWED_ORIGINS. Long-running work is offloaded to Celery workers, so requests return immediately with a job id. Job status moves through *created*, *queued*, *processing*, *completed*, *failed*. Alembic handles schema migrations.

8. Database / Storage Layer

Store	Holds
PostgreSQL 16	User (name, email), SourceContent (filename, storage path, content type), Job (prompt, formats, parameters, status, error), GeneratedArtefact (content, storage path, sha256_hash, guardrail_results). UUID string keys.
Qdrant	Chunk embeddings with job_id metadata (collection intelliforge_chunks).
MinIO (S3)	Original uploads and exported files.
Redis 7	Celery broker and result backend.

9. AI / ML Services

- **Base model:** Qwen2.5-7B-Instruct running locally; **embeddings:** BAAI/bge-m3. **ASR / OCR:** faster-whisper, Pytesseract.
- **Adapters:** QLoRA (4-bit, rank 16, learning rate 2e-4, 3 epochs) trained on synthetic pairs (200 per format targeted, generated with the Groq API during development only); hot-swapped per request through vLLM. Fallback: system-prompt engineering per format.
- **External services at runtime:** none in local mode. An optional LLM_MODE=cloud setting exists; cloud provider: Not specified in the provided content.

10. Security Architecture

Control	Implementation
Access	JWT / OAuth2 with role-based access (analyst, admin, auditor). CORS allow-list.
Input	File type and size validation; Presidio PII detection and redaction; rule-based prompt-injection detection; policy screen.
Output	Grounding (flag below 0.6), toxicity (flag above 0.5), plagiarism similarity and format checks. Grounding and toxicity block completion; plagiarism and format are warnings.
Integrity + audit	SHA-256 hash per artefact; guardrail actions logged; human approval before export. Blockchain audit ledger planned.
Privacy + secrets	Offline processing keeps data on the local machine; configuration and credentials in .env. Encryption at rest / in transit: Not specified in the provided content.

11. Deployment Architecture

Docker Compose runs PostgreSQL, Redis, MinIO and Qdrant. Three processes run alongside: the FastAPI server (uvicorn, port 8000), the Celery worker, and the Vite frontend. Inference uses an NVIDIA GPU (CUDA 12.x) with CPU fallback. Models, images and packages are downloaded once; afterwards the full pipeline runs offline. Cloud hosting details: Not specified in the provided content.

12. Scalability and Future Enhancements

- **Scalability:** Celery workers process jobs concurrently; vLLM serves several adapters from one base model; MinIO lifecycle rules can archive older versions.
- **Planned:** blockchain audit ledger (SHA-256 registry), fast-track guardrail mode (PII + factual grounding), LoRA adapters for all formats, entity extraction (currently an empty list), and comparison against external public text for plagiarism.